FitKV: Fitting the KV Cache Budget to Each Input for Long Context Inference

Paul Trust
trustpaul1000@gmail.com

Abstract

The key value (KV) cache of a large language model (LLM) grows linearly with the input and dominates memory in long context inference. Eviction methods reduce it by keeping the tokens that receive the most attention, but they give every input the same budget, although some inputs can be answered from a small part of their context and others need nearly all of it. We propose FitKV, a training free method that measures how strongly the final prompt queries agree on where to attend, and uses this agreement to give each input a budget between a user given minimum and its full length. On LongBench, with every comparison made on identical examples, FitKV outperforms fixed budget methods at small budgets on all three LLMs we test, and with a minimum of 128 tokens it retains at least 96% of full cache accuracy in every task category, compared with 52 to 93% for fixed budgets. An ablation attributes the gain to the input adaptive budget rather than to token selection.

1 Introduction

Large language models (LLMs) are increasingly used to read long inputs, such as reports, codebases, dialogue histories and collections of retrieved passages. While generating, every layer of the model stores a key and a value vector for each input token, and every new token reads this key value (KV) cache in full. For Qwen2.5-7B, the KV cache of a single 128K token input occupies 7 GiB in 16 bit precision, half the size of the model weights, so the cache rather than the computation limits how many requests can be served at once and how fast tokens are generated.

KV cache eviction reduces this cost by keeping only part of the cache once the prompt has been processed. The prevailing approach scores each cached token by the attention it receives and keeps the highest scoring tokens within a budget: H2O (Zhang et al., 2023) accumulates attention over all prompt positions, SnapKV (Li et al., 2024) uses a window of queries at the end of the prompt, and PyramidKV (Cai et al., 2024) and Ada-KV (Feng et al., 2025) redistribute the budget across layers and attention heads. In all of these methods, the total budget is the same for every input.

Inputs, however, differ widely in how much of their context they use. A question answered by a single sentence needs very little of its context, while a question whose evidence is spread over a long input needs much more, and both kinds occur within the same task. In the six question answering tasks we study, SnapKV with a budget of only 128 tokens already reproduces the full cache score for 60% of the examples, whereas for 20% even 2,048 tokens are not enough (Figure 1). A fixed budget is therefore too large for most inputs and too small for a sizeable minority, and no single value serves both. The difficulty is to tell the two kinds of input apart before generating, without labels and without an extra pass through the model.

We address this with FitKV, which derives the budget of each input from the attention its prompt already produces. After the prompt has been processed, FitKV examines the last third of the layers and asks how much the final prompt queries, which usually contain the question, agree on which parts of the context to read. When they agree, the relevant evidence is shared by the whole window and the model's full cache behaviour is worth preserving; when they disagree, attention is already fragmented and the cache can shrink towards the minimum, keeping the parts that individual queries need. This agreement is turned into a scalar between zero and one that places the budget of the input between a user given minimum and the full input length, and the same attention statistics then decide which tokens are kept (Figure 2). The method needs no training, no threshold and no extra model pass.

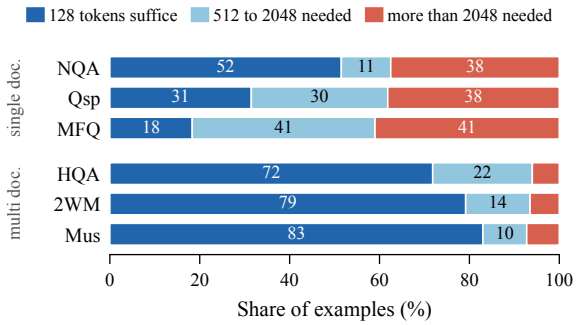


Figure 1: How much cache does an example need? For each example of the six question answering tasks, we find the smallest budget at which SnapKV reproduces the full cache score, pooled over Qwen2.5-3B and Qwen2.5-7B. In the single document tasks, 18 to 52% of examples need only 128 tokens while 38 to 41% need more than 2,048, so no fixed budget suits them.

We evaluate FitKV on LongBench (Bai et al., 2024) with three LLMs, five budgets and five eviction baselines. All comparisons are paired on identical examples, and every reported difference carries a bootstrap confidence interval. FitKV improves significantly on the best fixed budget method in ten of twelve settings and stays within 0.6 points of the full cache on the six question answering tasks at every budget, and an ablation that keeps its token selection but fixes the budget shows that the input adaptive budget accounts for the gain. Our contributions are an analysis showing that the cache an input needs varies widely within a task, a training free estimate of that need from attention agreement, and controlled evidence that adapting the budget per input closes most of the gap between small fixed budgets and the full cache.

2 Related Work

Reducing the KV cache after the prompt has been processed has become a standard way to make long context inference affordable, and most methods differ in how they decide which tokens matter. H2O (Zhang et al., 2023) keeps the tokens with the largest attention accumulated over all queries, on the premise that tokens attended to often in the past remain important. Under causal attention, early tokens are seen by more queries, so accumulated scores favour them, and computing the scores requires attention from every prompt position. SnapKV (Li et al., 2024) avoids both problems by pooling only the attention of a short observation window at the end of the prompt, which usually contains the question, and has become the standard base for later work. Quest (Tang et al., 2024) keeps the entire cache in memory and, at each generation step, loads only the blocks of tokens whose upper bound on attention is highest, which saves memory bandwidth but not memory. StreamingLLM (Xiao et al., 2024b) and InfLLM (Xiao et al., 2024a) target unbounded streams, keeping a few initial tokens and a recent window, or retrieving stored blocks on demand. Common to all of these methods is a single budget for every input, which our method keeps as a minimum but no longer imposes as a ceiling.

A second line of work changes how a fixed budget is distributed rather than which tokens fill it. PyramidKV (Cai et al., 2024) gives lower layers, whose attention is broad, a larger share than upper layers, whose attention is focused, and Ada-KV (Feng et al., 2025) lets heads with concentrated attention give up capacity to heads that need it. Both methods leave the total budget of an input unchanged, and both lose accuracy when that total is small. Our method is complementary to them: it changes the total per input, and either allocation scheme could be applied within the budget it assigns.

Attention weights alone ignore the content of the cached values; Guo et al. (2024) show that adding a value based term improves token selection, and our score includes a related term. ReFreeKV (Ni et al., 2026) also abandons a fixed budget: it prunes tokens in order of importance until the attention norm of the remaining cache drops by more than a fixed fraction, aiming at full cache accuracy with as few tokens as possible. Our method differs in the signal and in the guarantee: it reads the agreement among many prompt queries in the late layers rather than the attention of a single token, and it respects a minimum budget set by the user, so that compression is never weaker than the user requires for inputs with strong disagreement.

Like Quest, our method evicts contiguous blocks of tokens rather than single tokens. Blocks are a property of the cache, not of the input: they apply equally to documents, dialogues and code, and they map directly onto the fixed size memory blocks of serving systems such as PagedAttention (Kwon et al., 2023).

3 Method

3.1 Setting and notation

A decoder has L layers indexed by ℓ, each with H key value heads of dimension d. For models with grouped query attention (GQA; Ainslie et al., 2023), we average the queries within each group so that every query corresponds to one key value head. After processing a prompt x of n tokens, we hold the queries $\boldsymbol{q}_t^{(\ell,h)}$, taken after the rotary position embedding, and the cached keys $\boldsymbol{k}_i^{(\ell,h)}$ and values $\boldsymbol{v}_i^{(\ell,h)}$ of every layer ℓ and head h. We group the cache into blocks of P consecutive tokens, the unit of eviction. Let $\mathcal{W}$ be the last w positions of the prompt, the observation window, and let the context be the $n' = P\lfloor (n-w)/P \rfloor$ tokens before it, rounded down to whole blocks and split into $m = n'/P$ blocks $\mathcal{C}_1, \dots, \mathcal{C}_m$. The user specifies a minimum budget B, in tokens per layer. An eviction method keeps B_x tokens per layer for input x, with $B \le B_x \le n$, and discards the rest before generation; fixed budget methods use $B_x = B$ for every input.

3.2 Query agreement

We measure agreement through the attention that the window queries pay to the blocks of the context. For a query $t \in \mathcal{W}$ and a context token $i \le n'$, the attention weight is

$$a_{t,i}^{(\ell,h)} = \frac{\exp\big(\boldsymbol{q}_t^{(\ell,h)\top} \boldsymbol{k}_i^{(\ell,h)} / \sqrt{d}\big)}{\sum_{i' \le n'} \exp\big(\boldsymbol{q}_t^{(\ell,h)\top} \boldsymbol{k}_{i'}^{(\ell,h)} / \sqrt{d}\big)}. \quad (1)$$

Following SnapKV, we smooth $a_{t,i}^{(\ell,h)}$ along i with a moving average of width five, giving $\tilde{a}_{t,i}^{(\ell,h)}$, and aggregate it to blocks and heads:

$$A_{t,j}^{(\ell)} = \frac{1}{H} \sum_{h=1}^{H} \sum_{i \in \mathcal{C}_j} \tilde{a}_{t,i}^{(\ell,h)}. \quad (2)$$

Each row $A_{t,\cdot}^{(\ell)}$ describes where query t looks. Across the window, each block j thus receives a mean attention and a standard deviation,

$$\mu_j^{(\ell)} = \frac{1}{w} \sum_{t \in \mathcal{W}} A_{t,j}^{(\ell)},$$
$$\sigma_j^{(\ell)} = \Big(\frac{1}{w-1} \sum_{t \in \mathcal{W}} \big(A_{t,j}^{(\ell)} - \mu_j^{(\ell)}\big)^2 \Big)^{1/2}. \quad (3)$$

A block that all window queries read to the same extent has $\sigma_j^{(\ell)}$ close to zero, whereas a block that some queries read and others ignore has a large $\sigma_j^{(\ell)}$. We summarise a layer by its disagreement

$$D_\ell = \frac{\bar{\sigma}^{(\ell)}}{\bar{\mu}^{(\ell)} + \varepsilon}, \quad (4)$$

where $\bar{\sigma}^{(\ell)}$ and $\bar{\mu}^{(\ell)}$ are the averages of $\sigma_j^{(\ell)}$ and $\mu_j^{(\ell)}$ over the m blocks and $\varepsilon = 10^{-8}$ prevents division by zero. Because each row of $A^{(\ell)}$ sums to approximately one, $\bar{\mu}^{(\ell)} \approx 1/m$, and D_ℓ is a coefficient of variation that does not depend on the number of blocks. It is low when the window queries agree on where to look, whether their attention is broad or focused, and high when different queries look at different parts of the context.

We read the signal from the late layers, whose representations are closest to the output, and average the disagreement over the last third of them,

$$\bar{D} = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} D_\ell, \qquad \mathcal{L} = \{\lfloor 2L/3 \rfloor + 1, \dots, L\}. \quad (5)$$

Section 5 compares this choice with other sets of layers.

3.3 Input adaptive budget

The averaged disagreement is mapped to the unit interval by

$$\lambda = \min\big(1,\ \bar{D}/2\big), \quad (6)$$

where the scale of one half was fixed by hand and not tuned. The budget of input x is then

$$B_x = \big\lfloor\, n - \lambda\,(n - B)\,\big\rfloor. \quad (7)$$

Equation (7) starts from the full input and removes a fraction λ of the tokens above the minimum: with full agreement ($\lambda = 0$) the whole cache is kept, and with strong disagreement ($\lambda = 1$) the cache shrinks to the minimum B. For $n > B$ the budget always satisfies $B \le B_x \le n$ and decreases monotonically in λ, and the full cache is kept exactly when $\lambda(n - B) < 1$. As an example, consider an input of $n = 8{,}000$ tokens with a minimum of $B = 128$. If the late layer queries agree strongly, say $\bar{D} = 0.2$, then $\lambda = 0.1$ and the input keeps $B_x = 7{,}212$ tokens, about 90% of its cache; if they disagree, say $\bar{D} = 1.6$, then $\lambda = 0.8$ and it keeps 1,702 tokens; with $\bar{D} \ge 2$ it keeps only the minimum of 128. The minimum B thus bounds the cache of inputs with strong disagreement from below, while λ decides how far each input stays

above it. Fixing $\lambda = 1$ for every input turns FitKV into a fixed budget method that uses our token selection; we report this variant as FitKV (fixed B) to separate the contribution of the adaptive budget from that of the selection.

3.4 Block selection

Given B_x, every layer keeps the $K = \lfloor B_x/P \rfloor$ blocks with the highest scores. The score combines three terms computed from quantities already available. The first two are the mean and the standard deviation of Eq. (3), weighted by the layer's own disagreement $\lambda_\ell = \min(1, D_\ell/2)$, so that a block read by many queries, or read strongly by a few, scores highly. The third term measures how much a block's values would change the attention output. For head h, let $\bar{a}_j^{(\ell,h)}$ be the attention that block j receives, averaged over the window, let $\bar{\boldsymbol{v}}_j^{(\ell,h)}$ be the mean value vector of the block, and let $\bar{\boldsymbol{o}}^{(\ell,h)}$ be the attention output averaged over the window:

$$\bar{a}_j^{(\ell,h)} = \frac{1}{w} \sum_{t \in \mathcal{W}} \sum_{i \in \mathcal{C}_j} \tilde{a}_{t,i}^{(\ell,h)},$$

$$\bar{\boldsymbol{o}}^{(\ell,h)} = \frac{1}{w} \sum_{t \in \mathcal{W}} \sum_{i \le n'} \tilde{a}_{t,i}^{(\ell,h)} \, \boldsymbol{v}_i^{(\ell,h)}.$$

The value term is

$$g_j^{(\ell)} = \frac{1}{H} \sum_{h=1}^{H} \bar{a}_j^{(\ell,h)} \, \frac{\left\| \bar{\boldsymbol{v}}_j^{(\ell,h)} - \bar{\boldsymbol{o}}^{(\ell,h)} \right\|_2}{\sqrt{d}}, \qquad (8)$$

which is large for blocks that are attended to and whose values differ from what the window already reads. After rescaling $g^{(\ell)}$ to the range of the attention terms,

$$\tilde{g}_j^{(\ell)} = g_j^{(\ell)} \cdot \frac{\max_{j'} \left(\mu_{j'}^{(\ell)} + \lambda_\ell \, \sigma_{j'}^{(\ell)} \right)}{\max_{j'} g_{j'}^{(\ell)}}, \qquad (9)$$

the score of block j in layer ℓ is

$$s_j^{(\ell)} = \mu_j^{(\ell)} + \lambda_\ell \, \sigma_j^{(\ell)} + \eta \, \tilde{g}_j^{(\ell)}, \qquad (10)$$

with $\eta = 0.3$. The blocks of the observation window and the last two blocks of the context are always kept, and all heads of a layer keep the same blocks. Algorithm 1 gives the complete procedure.

The cost of FitKV is small compared with the prompt pass. Equation (1) restricted to the window takes $O(w\, n'\, d)$ operations per head and layer, as in SnapKV, and the value term adds operations of the same order; with $w = 64$, both are negligible next to the $O(n^2 d)$ cost of processing the prompt, and no additional pass through the model is required. We did not measure wall clock overhead.

Algorithm 1 FitKV, applied once after the prompt

Require: queries $\boldsymbol{q}$ and KV cache $(\boldsymbol{k}, \boldsymbol{v})$ of a prompt with n tokens; minimum budget B; block size P; window size w; value weight η
Ensure: KV cache with $\lfloor B_x/P \rfloor$ blocks per layer

if $n \le B$ **then**
  return $(\boldsymbol{k}, \boldsymbol{v})$ {nothing to evict}
end if
$n' \leftarrow P\lfloor (n-w)/P \rfloor$; $m \leftarrow n'/P$
for $\ell = 1, \ldots, L$ **do**
  $A^{(\ell)} \leftarrow$ block attention, Eqs. (1) and (2)
  $\mu^{(\ell)}, \sigma^{(\ell)} \leftarrow$ Eq. (3); $D_\ell \leftarrow$ Eq. (4)
  $\lambda_\ell \leftarrow \min(1, D_\ell/2)$
end for
$\lambda \leftarrow \min\big(1, \bar{D}/2\big)$ with $\bar{D}$ from Eq. (5)
$B_x \leftarrow \lfloor n - \lambda(n-B) \rfloor$ {Eq. (7)}
if $B_x = n$ **then**
  return $(\boldsymbol{k}, \boldsymbol{v})$ {keep the full cache}
end if
$K \leftarrow \lfloor B_x/P \rfloor$
for $\ell = 1, \ldots, L$ **do**
  $\tilde{g}^{(\ell)} \leftarrow$ Eqs. (8) and (9)
  $s_j^{(\ell)} \leftarrow \mu_j^{(\ell)} + \lambda_\ell \sigma_j^{(\ell)} + \eta \, \tilde{g}_j^{(\ell)}$ for all j
  $s_j^{(\ell)} \leftarrow \infty$ for window blocks and the last two blocks
  keep the K blocks with the largest $s_j^{(\ell)}$; evict the others
end for
return the compressed cache

4 Experimental Setup

Models. We use the base models Qwen2.5-3B and Qwen2.5-7B (Yang et al., 2024) and Llama-3.1-8B (Grattafiori et al., 2024), and the instruction tuned Phi-3.5-mini (Abdin et al., 2024). Qwen2.5-3B has 36 layers with 2 key value heads per layer, Qwen2.5-7B has 28 layers with 4, and Llama-3.1-8B has 32 layers with 8; all three use GQA. Phi-3.5-mini has 32 layers with standard multi head attention. Prompts follow the LongBench templates, inputs are truncated to 16,384 tokens, models run in 16 bit floating point, and decoding is greedy.

Tasks and metrics. Our main measure is the mean score over six question answering tasks, three on single documents, NarrativeQA (NQA), Qasper (Qsp) and MultiFieldQA (MFQ), and three on multiple documents, HotpotQA (HQA), 2WikiMultihopQA (2WM) and MuSiQue (Mus), all scored by token F1. Where all methods have matched runs, we add the few shot tasks TREC

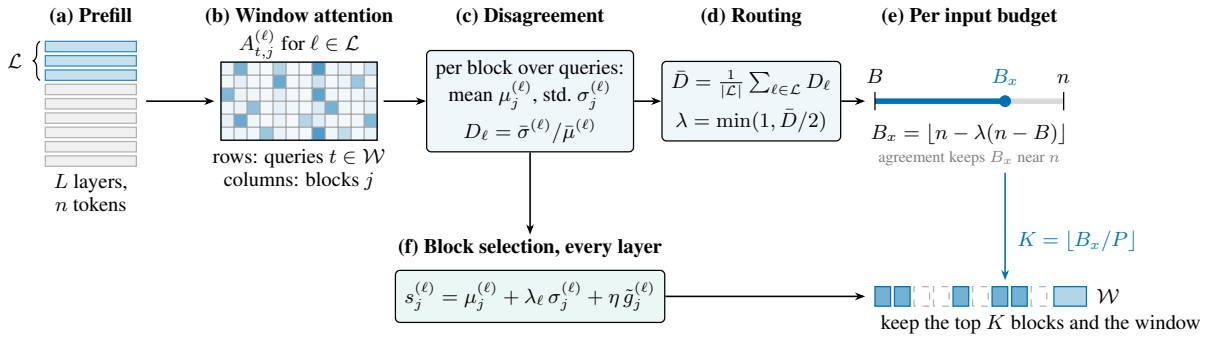


Figure 2: Overview of FitKV. (a) After the prompt pass over n tokens, the queries and the KV cache of every layer are available; the last third of the layers forms $\mathcal{L}$. (b) In each late layer, the window queries $t \in \mathcal{W}$ attend to the context blocks j. (c) Each block receives a mean $\mu^{(\ell)}_j$ and a standard deviation $\sigma^{(\ell)}_j$ of attention over the queries, and their ratio D_ℓ measures how much the queries disagree on where to look. (d) The late layer average $\bar{D}$ gives the routing scalar λ. (e) λ places the budget B_x of the input between the minimum B and the full length n (Eq. 7). (f) Every layer scores its blocks with the same statistics and a value term (Eq. 10) and keeps the top $K = \lfloor B_x/P \rfloor$ blocks together with the window.

(classification accuracy) and TriviaQA (TQA, F1), and the synthetic tasks PassageCount (PCnt) and PassageRetrieval (PRe), both scored by accuracy. Summarisation and code tasks score at or near zero even with the full cache in our setup and are excluded. Scores are multiplied by 100.

Matched comparisons. LongBench provides two versions of most tasks, the original and the length balanced LongBench-E, whose example sets differ. To keep every comparison paired, we compare methods only on identical examples, which yields two groups of settings. Grid A covers Qwen2.5-3B and Qwen2.5-7B at $B \in \{128, 512, 2048\}$ on the six question answering tasks, for all methods and the full cache. Grid B covers $B \in \{256, 1024\}$ for Qwen2.5-3B, Qwen2.5-7B and Llama-3.1-8B on the tasks where H2O, Quest, SnapKV and both variants of our method share examples, listed in Table 2; PyramidKV and Ada-KV were run on different examples for several of these tasks and are omitted there. Phi-3.5-mini enters only the layer ablation, the one experiment in which all its runs are matched.

Baselines and implementation. We compare with H2O, Quest, SnapKV, PyramidKV and Ada-KV, and with FitKV (fixed B). All methods share one implementation with blocks of $P = 16$ tokens, a window of $w = 64$ queries, smoothing width five and the last two blocks always kept, so differences come only from token selection and budget. H2O always keeps the first four tokens, PyramidKV lets the budget decrease linearly across layers with the smallest share equal to the average divided by 20, and Ada-KV guarantees each head at least 20% of its uniform share, following the original papers. Quest is applied once after the prompt, with its attention upper bound as the block score, rather than at every generation step.

Statistics. B is the per layer budget of fixed budget methods and the minimum budget of FitKV. Each setting is a single greedy run. For each difference between two methods we report a paired bootstrap interval (Koehn, 2004), resampling examples with replacement within each task 2,000 times and taking the 2.5 and 97.5 percentiles of the difference in mean score. Two separate full cache runs of Qwen2.5-7B agree on 98.3% of examples and differ by 0.13 points on average, which indicates the size of run to run variation.

5 Results and Discussion

Accuracy across budgets. Table 1 and Figure 3 show that fixed budget methods lose accuracy steadily as the budget shrinks. At $B = 128$, the best of them trails the full cache by 7.3 points on Qwen2.5-3B (Ada-KV, 19.1 against 26.4) and by 4.2 points on Qwen2.5-7B (FitKV (fixed B), 14.4 against 18.6), and the gap closes only at $B = 2048$. H2O and Quest stay far below the other methods at every budget, since accumulated attention and single query bounds select poorly when the cache is reduced once after the prompt. FitKV stays within 0.6 points of the full cache at every budget on both models, which at $B = 128$ amounts to an improvement of 6.8 points on Qwen2.5-3B and 4.3 points

Method	Qwen2.5-3B			Qwen2.5-7B		
	128	512	2048	128	512	2048
Full cache	*26.4*	*26.4*	*26.4*	*18.6*	*18.6*	*18.6*
H2O	10.6	13.8	15.9	10.6	12.2	14.3
Quest	11.6	14.5	16.6	12.3	14.4	15.9
SnapKV	18.8	23.6	**26.2**	14.3	17.3	18.1
PyramidKV	17.6	20.0	24.0	13.5	15.8	18.0
Ada-KV	19.1	22.3	26.0	13.9	15.5	18.4
FitKV (fixed B)	18.8	24.2	25.9	14.4	17.3	18.5
FitKV	**25.9**	**25.9**	26.1	**18.6**	**18.6**	**18.7**

Table 1: Grid A: mean score over the six question answering tasks for budgets B of 128, 512 and 2048 tokens per layer (higher is better). All rows of a column use identical examples. **Bold**: best compressed method; underline: second best.

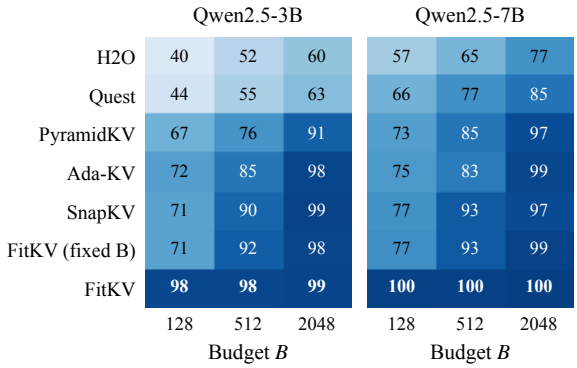


Figure 3: Grid A as the percentage of full cache accuracy that each method retains, averaged over the six question answering tasks. FitKV is the only method that retains at least 98% at every budget.

on Qwen2.5-7B over the best fixed budget method.

Other models and tasks. Table 2 extends the comparison to $B \in \{256, 1024\}$, to Llama-3.1-8B, and to few shot and synthetic tasks. FitKV has the highest mean in all six settings, and its margin over the best baseline is largest at $B = 256$, with 6.2 points on Qwen2.5-3B, 9.2 on Qwen2.5-7B and 4.0 on Llama-3.1-8B, shrinking to between 1.2 and 1.7 points at $B = 1024$. The three models differ in size, number of key value heads and training data, yet FitKV uses the same hyperparameters for all of them.

Statistical reliability. Table 3 lists the paired differences for all twelve settings. FitKV is significantly better than the best baseline in ten of them; the two exceptions are at $B = 2048$, where fixed budgets already approach the full cache and the differences of -0.1 and $+0.4$ points fall inside their intervals. Every significant gain exceeds the

Method	Qwen2.5-3B		Qwen2.5-7B		Llama-3.1-8B	
	256	1024	256	1024	256	1024
Tasks	10	7	5	7	8	8
H2O	16.0	25.1	24.3	23.7	20.2	22.4
Quest	17.9	26.5	30.1	27.8	20.1	22.0
SnapKV	22.6	36.1	42.1	38.4	24.5	27.4
FitKV (fixed B)	23.1	36.4	42.0	38.5	24.7	27.6
FitKV	**28.8**	**37.8**	**51.4**	**39.6**	**28.6**	**28.6**

Table 2: Grid B: mean score over the tasks on which all listed methods share identical examples, for B of 256 and 1024; formatting as in Table 1. Tasks: Qwen2.5-3B, $B = 256$: all six QA tasks, TREC, TQA, PCnt, PRe; $B = 1024$: Qsp, MFQ, HQA, 2WM, Mus, TREC, TQA. Qwen2.5-7B, $B = 256$: MFQ, TREC, TQA, PCnt, PRe; $B = 1024$: NQA, MFQ, Mus, TREC, TQA, PCnt, PRe. Llama-3.1-8B, both budgets: all six QA tasks, TREC, TQA.

Model	B	Tasks	vs. best baseline	vs. fixed B
Qwen2.5-3B	128	6	+6.8 [+5.5, +8.0]	+7.1 [+5.8, +8.3]
	256	10	+6.2 [+5.2, +7.2]	+5.6 [+4.7, +6.6]
	512	6	+2.2 [+1.3, +3.2]	+1.7 [+0.9, +2.5]
	1024	7	+1.7 [+0.9, +2.5]	+1.4 [+0.6, +2.2]
	2048	6	−0.1 [−0.7, +0.5]	+0.2 [−0.4, +0.7]
Qwen2.5-7B	128	6	+4.3 [+3.2, +5.4]	+4.1 [+3.1, +5.2]
	256	5	+9.2 [+7.5, +11.0]	+9.4 [+7.7, +11.2]
	512	6	+1.3 [+0.4, +2.2]	+1.3 [+0.4, +2.1]
	1024	7	+1.3 [+0.5, +2.0]	+1.1 [+0.3, +1.8]
	2048	6	+0.4 [−0.4, +1.1]	+0.2 [−0.3, +0.8]
Llama-3.1-8B	256	8	+4.0 [+3.3, +4.8]	+3.8 [+3.1, +4.6]
	1024	8	+1.2 [+0.7, +1.8]	+0.9 [+0.4, +1.4]

Table 3: Difference in mean score (points) between FitKV and the best baseline of each setting (left) or FitKV (fixed B) (right), with 95% paired bootstrap intervals. Bold: interval entirely above zero. Settings with six tasks belong to Grid A, the others to Grid B.

run to run variation of 0.13 points by a wide margin.

Task categories. Table 4 shows that at $B = 128$ single document QA suffers most under fixed budgets, which retain only 52 to 67% of full cache accuracy, while multi document QA retains 80 to 93% and few shot tasks 62 to 76%; FitKV retains between 96.8 and 101.5% in every category on both models. Figure 1 explains the pattern: in single document tasks, where the answer to a question about a long article may lie anywhere in it, the cache an example needs varies most from one example to the next. On individual tasks, SnapKV loses 17.5 points on MFQ and 13.0 points on Qsp with Qwen2.5-3B at $B = 128$, whereas FitKV loses at most 1.6 points on any of the six question answering tasks with either model. NQA and Mus have low scores for every method, including the full cache, partly because many NQA inputs ex-

Method	Qwen2.5-3B			Qwen2.5-7B		
	Single	Multi	Few	Single	Multi	Few
SnapKV	59.6	82.1	75.9	66.0	87.7	63.2
PyramidKV	52.6	80.0	73.7	52.2	93.1	61.5
Ada-KV	60.6	83.9	75.8	57.9	91.7	62.6
FitKV (fixed B)	59.7	82.6	76.3	67.3	87.7	63.2
FitKV	**96.8**	**99.5**	**100.0**	**97.9**	**101.5**	**98.9**

Table 4: Accuracy retained at $B = 128$, in percent of the full cache score, for single document QA (NQA, Qsp, MFQ), multi document QA (HQA, 2WM, Mus) and few shot tasks (TREC and TQA; TREC only for Qwen2.5-7B). Values above 100 mean that the compressed cache scored higher than the full cache.

ceed the 16,384 token limit and lose their question to truncation, so they contribute little to the differences. The largest single gains appear on TREC, a few shot classification task whose labelled examples are spread over the prompt: at $B = 256$, the best baseline reaches 49.0 on Qwen2.5-3B, 46.7 on Qwen2.5-7B and 50.7 on Llama-3.1-8B, and FitKV 69.0, 71.5 and 66.0. PRe behaves similarly on Qwen2.5-7B at $B = 256$ (80.0 against 67.0), whereas TQA, whose answer is usually in one short passage, and PCnt, which every method solves poorly, show no meaningful differences. The gains are therefore largest on tasks whose evidence is spread across the input.

Effect on individual examples. Mean scores can hide large changes that cancel out, so Figure 4 compares every example with its full cache score. At $B = 128$, fixed budget methods lower the score of 30 to 41% of examples and raise it for 10 to 14%, whereas FitKV lowers 6.5% and raises 6.4%, proportions that barely change between $B = 128$ and $B = 2048$. Its distribution at $B = 128$ is closer to the full cache than that of SnapKV at $B = 2048$, which still lowers 12.3% of scores. That eviction sometimes raises a score agrees with the view that removing redundant context can sharpen attention on the evidence, but for fixed budgets the losses dominate: summed over the examples, SnapKV gains 58.9 F1 units and loses 210.5 at $B = 128$. FitKV gains 12.6 and loses 17.7, so its mean stays close to the full cache because it rarely changes an answer, not because gains and losses cancel. Of the examples that SnapKV still gets wrong at $B = 2048$ (168 for Qwen2.5-3B and 169 for Qwen2.5-7B), FitKV reproduces the full cache score on 32.1% and 52.1% with a sixteen times smaller minimum, which suggests that its budget rises well above B for such inputs.

Remaining errors. FitKV still lowers the score of some examples, and these losses are concentrated. At $B = 128$, 12.7 to 14.0% of the NQA, Qsp and MFQ examples score below the full cache, against at most 1.8% on each multi document task, and the same three single document tasks account for most of the remaining loss at $B = 2048$. In 81% of these cases SnapKV at the same budget fails as well, so the errors arise on inputs whose evidence a 128 token cache cannot hold and for which the routing signal still assigned a small budget. For questions over a single long document, the window queries can agree on a few blocks that look relevant while the answer lies elsewhere, so agreement is a weaker indicator of sufficiency there than on multi document inputs. A signal that also accounts for the length of the document, or that is refined after the first generated token, could reduce these errors.

Reallocating a budget versus adapting it. PyramidKV and Ada-KV redistribute a fixed budget, across layers and across heads, using the same SnapKV scores, yet in Grid A neither improves consistently on SnapKV: Ada-KV is between 1.8 points below and 0.4 points above it, and PyramidKV is below it in all six settings, by up to 3.7 points. Redistributing a small budget cannot recover evidence that the budget is too small to hold, and at $B = 128$ all three remain more than 4 points below the full cache on both models. Adapting the total budget per input targets this shortfall, and FitKV gains most at the budgets where reallocation helps least.

Where the gain comes from. FitKV (fixed B) uses the token selection of Eq. (10) with a fixed budget. In Grid A it is within 0.6 points of SnapKV in all six settings, so the disagreement and value terms add little on their own. Adding the input adaptive budget improves on FitKV (fixed B) in ten of twelve settings (Table 3, right): by 7.1 and 4.1 points at $B = 128$, by 3.8 to 9.4 points at $B = 256$, and by 0.9 to 1.7 points at $B \in \{512, 1024\}$. At $B = 2048$ the difference is within noise, as expected when the minimum itself approaches the length most inputs need. The gain of FitKV therefore comes from deciding how much to keep rather than which tokens to keep.

Sensitivity to design choices. A fixed budget method must be tuned to the workload, because

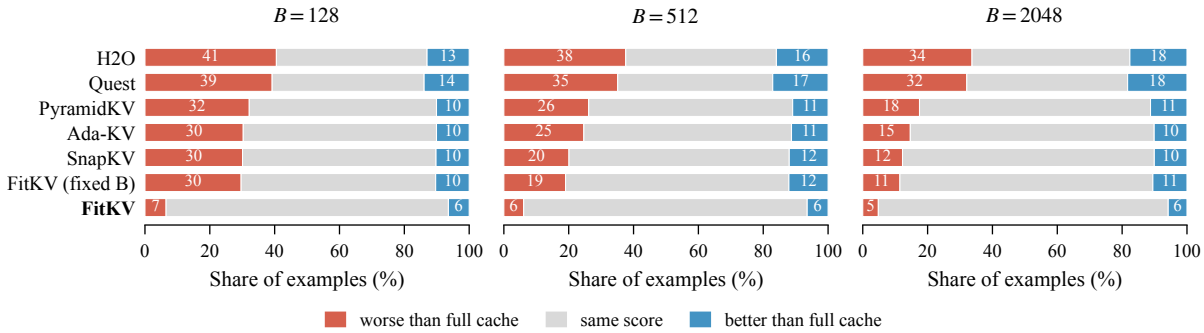


Figure 4: Effect of eviction on individual examples in Grid A, pooled over Qwen2.5-3B, Qwen2.5-7B and the six question answering tasks (2,748 examples per budget). Each bar splits the examples into those scored lower than with the full cache (red, with percentage), the same (grey) and higher (blue, with percentage).

	Qwen2.5-7B		Phi-3.5-mini	
Layers for λ	256	1024	256	1024
First third	15.24	15.33	13.66	13.62
Middle third	15.32	15.17	13.61	13.57
All layers	15.36	15.25	13.60	13.65
Last layer	15.12	15.18	13.59	13.67
Last third (default)	15.24	15.34	13.62	13.59

Table 5: Effect of the layers used to compute $\bar{D}$ (Eq. 5): mean score over NQA, Qsp, HQA, 2WM and Mus, with identical examples in each column.

its accuracy depends strongly on B. FitKV is much less sensitive: across Grid A, its output at $B = 128$ is identical to its output at $B = 2048$ for 90.9% of examples on Qwen2.5-3B and 91.9% on Qwen2.5-7B, and its mean at $B = 128$ (25.9 and 18.6) is comparable to the best fixed budget method at $B = 2048$ (26.2 and 18.5). The minimum therefore acts as a safeguard for inputs with strong disagreement rather than as a parameter to be matched to the task. Table 5 varies the layers in $\mathcal{L}$, using the first, middle or last third, all layers, or the last layer alone. On Qwen2.5-7B and Phi-3.5-mini, at $B = 256$ and $B = 1024$, no variant differs from the default by more than 0.2 points and every paired interval contains zero; on Qwen2.5-7B all variants are also within 0.25 points of the full cache (15.15). The budget is thus robust to the choice of layers, although our data cannot show that late layers are strictly better. FitKV is therefore most useful when the memory budget is small relative to typical inputs and when inputs of the same task differ in how much of their context they use; when the budget is already large, fixed and adaptive budgets perform alike.

6 Conclusion

We showed that the KV cache an input needs varies widely within a task, so that any fixed budget is too large for most inputs and too small for some. FitKV sets the budget of each input from the agreement of the late layer attention of its final prompt queries, between a user given minimum and the full input, without training or thresholds. In matched comparisons on three LLMs and five budgets, it improves significantly on the best fixed budget method in ten of twelve settings and retains at least 96% of full cache accuracy in every task category at a minimum of 128 tokens on both Qwen2.5 models, and an ablation attributes this gain to the adaptive budget rather than to token selection. Recording the budget assigned to each input would allow accuracy to be traded explicitly against average memory.

Limitations

FitKV keeps more than the minimum B for some inputs, and our runs did not record how many tokens each input kept, so we compare methods at the same minimum rather than the same average memory and make no claim about memory savings or latency. Our results cover question answering, few shot and synthetic LongBench tasks on models of up to 8B parameters, use single runs and our own reimplementations of the baselines, and rely on hand set values for the scale in Eq. (6) and the set of late layers.

References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, et al. 2024. Phi-3 technical report: A highly capable language

model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*.

Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *Advances in Neural Information Processing Systems*.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Zhiyu Guo, Hidetaka Kamigaito, and Taro Watanabe. 2024. Attention score is not all you need for token importance indicator in KV cache reduction: Value also matters. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21158–21166.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*.

Xuanfan Ni, Liyan Xu, Chenyang Lyu, Longyue Wang, Mo Yu, Lemao Liu, Fandong Meng, Jie Zhou, and Piji Li. 2026. ReFreeKV: Towards threshold-free KV cache compression. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*.

Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024a. InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory. In *Advances in Neural Information Processing Systems*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024b. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*.

A Implementation Details

This appendix collects the information needed to reproduce our runs in one place. Table 6 lists every hyperparameter of FitKV, of the baselines and of inference, and Table 7 summarises the notation of Section 3. All methods, including the baselines, run in the same code base and differ only in the block score and, for FitKV, in the budget.

Eviction is applied once, after the prompt has been processed and before the first token is generated; generated tokens are appended to the compressed cache without further eviction. The maximum number of generated tokens is 128 for NQA and Qsp, 64 for MFQ and TREC, and 32 for the other tasks, and for TREC and TQA only the first generated line is scored. A layer contributes to $\bar{D}$ only if its context is at least one block longer than the window and the window contains at least two queries, so that the standard deviation is defined; if no late layer qualifies, which happens only for very short prompts, λ is set to 0.5. The code, the per example scores of every run and the script that builds all tables and figures from them are part of the release.

Setting	Value
Block size P	16 tokens
Window size w	64 queries
Smoothing width	5 tokens
Always kept	window, last 2 blocks
Late layers $\mathcal{L}$	$\lfloor 2L/3 \rfloor + 1, \ldots, L$
Scale in Eq. (6)	$1/2$
Value weight η	0.3
ε	10^{-8}
H2O initial tokens kept	4
PyramidKV smallest share	average / 20
Ada-KV minimum head share	20%
Maximum input	16,384 tokens
Precision	16 bit floating point
Decoding	greedy, batch size 1

Table 6: Hyperparameters of FitKV (top), of the baselines (middle) and of inference (bottom).

B Per Task Results

The main text reports averages over tasks; this appendix gives the score of every method on every task, so that each average can be traced and per task effects can be checked. Each table corresponds to one model and one budget, and its last row gives the number of examples N per task, which is the same for every method in the table. Tables 8 to 13 belong to Grid A and include the full cache; at $B = 128$ they also include the few shot tasks that were run on matched examples for every method, and the full cache score for TREC on Qwen2.5-7B comes from a separate full cache run on the same examples. Tables 14 to 19 belong to Grid B and are restricted to the tasks on which all listed methods share examples, which is why their columns differ. Three observations stand out. First, at small budgets the losses of fixed budget methods concentrate on Qsp and MFQ, whereas FitKV stays within 1.6 points of the full cache on every question answering task. Second, NQA and Mus have low scores for every method, including the full cache, so they contribute little to the differences between methods. Third, on TREC every fixed budget method falls by more than 20 points at $B \leq 256$, while FitKV remains close to the full cache.

Symbol	Meaning
L, ℓ	number of layers, layer index
H, h	key value heads per layer, head index
d	head dimension
n, n'	prompt length, context length before the window
P, m, $\mathcal{C}_j$	block size, number of blocks, block j
$\mathcal{W}$, w	observation window, its size
$A_{t,j}^{(\ell)}$	attention of query t to block j
$\mu_j^{(\ell)}$, $\sigma_j^{(\ell)}$	mean and std. of $A_{t,j}^{(\ell)}$ over $t \in \mathcal{W}$
D_ℓ, $\bar{D}$	disagreement of layer ℓ, late layer average
$\mathcal{L}$	set of late layers
λ, λ_ℓ	budget scalar, layer weight in the score
B, B_x	minimum budget, budget of input x
$g_j^{(\ell)}$, $\tilde{g}_j^{(\ell)}$, η	value term, rescaled value term, weight
$s_j^{(\ell)}$, K	block score, blocks kept per layer

Table 7: Notation. Every symbol keeps this meaning throughout the paper.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	TREC	TQA	Avg.
Full cache	*5.6*	*25.8*	*46.0*	*39.3*	*35.6*	*6.0*	*60.7*	*65.1*	*35.5*
H2O	3.0	9.7	16.0	17.0	15.8	2.2	34.4	42.0	17.5
Quest	2.4	10.7	19.5	18.8	16.1	1.9	33.8	48.0	18.9
SnapKV	4.8	12.8	28.4	31.5	30.3	4.7	36.3	59.1	26.0
PyramidKV	4.3	10.8	25.6	29.5	30.4	4.8	35.7	57.0	24.8
Ada-KV	4.9	12.8	29.2	32.2	30.7	5.0	36.0	59.3	26.2
FitKV (fixed B)	5.3	12.9	28.0	31.6	30.6	4.6	36.3	59.6	26.1
FitKV	5.8	24.1	44.9	39.4	35.1	6.0	60.7	65.0	35.1
N	200	224	150	300	300	200	300	300	

Table 8: Per task scores, Qwen2.5-3B, $B = 128$. All rows use identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	Avg.
Full cache	*5.6*	*25.8*	*46.0*	*39.3*	*35.6*	*6.0*	*26.4*
H2O	3.6	13.5	20.3	21.4	20.6	3.2	13.8
Quest	3.2	15.6	24.1	20.8	20.1	3.0	14.5
SnapKV	5.9	18.9	42.2	37.0	32.3	5.5	23.6
PyramidKV	5.4	15.4	32.0	31.4	31.1	4.6	20.0
Ada-KV	4.5	17.3	40.5	35.7	31.3	4.6	22.3
FitKV (fixed B)	6.5	21.3	42.3	37.6	31.9	5.5	24.2
FitKV	5.8	24.1	44.7	39.2	35.4	6.0	25.9
N	200	224	150	300	300	200	

Table 9: Per task scores, Qwen2.5-3B, $B = 512$. All rows use identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	Avg.
Full cache	*5.6*	*25.8*	*46.0*	*39.3*	*35.6*	*6.0*	*26.4*
H2O	4.0	18.7	25.4	21.9	21.6	3.7	15.9
Quest	4.0	20.3	27.1	21.9	21.9	4.2	16.6
SnapKV	6.1	23.6	45.0	40.0	35.3	7.2	26.2
PyramidKV	5.3	20.2	40.1	38.5	33.5	6.7	24.0
Ada-KV	4.6	22.4	45.1	40.3	35.8	7.5	26.0
FitKV (fixed B)	5.7	22.9	44.4	40.2	34.8	7.5	25.9
FitKV	5.7	24.5	45.7	39.4	35.2	6.0	26.1
N	200	224	150	300	300	200	

Table 10: Per task scores, Qwen2.5-3B, $B = 2048$. All rows use identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	TREC	Avg.
Full cache	*6.1*	*13.8*	*36.1*	*31.5*	*18.7*	*5.6*	*60.7*	*24.6*
H2O	3.3	9.7	16.3	17.6	13.1	3.5	36.0	14.2
Quest	3.6	12.6	20.8	20.0	14.1	2.4	35.3	15.6
SnapKV	5.3	7.2	24.5	28.1	16.9	3.9	38.3	17.8
PyramidKV	5.3	6.6	17.4	29.4	18.6	4.1	37.3	16.9
Ada-KV	5.5	7.1	19.9	29.3	16.8	5.1	38.0	17.4
FitKV (fixed B)	5.5	7.1	25.1	28.6	16.4	3.9	38.3	17.9
FitKV	6.1	13.9	34.8	31.6	19.4	5.7	60.0	24.5
N	200	224	150	300	300	200	300	

Table 11: Per task scores, Qwen2.5-7B, $B = 128$. All rows use identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	Avg.
Full cache	*6.1*	*13.8*	*36.1*	*31.5*	*18.7*	*5.6*	*18.6*
H2O	4.0	14.3	21.2	17.8	12.9	2.9	12.2
Quest	4.4	17.3	25.5	22.3	14.5	2.6	14.4
SnapKV	5.4	10.6	33.5	34.3	15.9	4.0	17.3
PyramidKV	5.4	8.6	29.7	30.6	15.8	4.8	15.8
Ada-KV	6.0	8.4	26.6	30.6	17.1	4.3	15.5
FitKV (fixed B)	5.6	10.7	33.2	34.1	15.9	4.5	17.3
FitKV	6.2	13.7	35.1	31.6	19.3	5.7	18.6
N	200	224	150	300	300	200	

Table 12: Per task scores, Qwen2.5-7B, $B = 512$. All rows use identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	Avg.
Full cache	*6.1*	*13.8*	*36.1*	*31.5*	*18.7*	*5.6*	*18.6*
H2O	4.5	18.5	24.5	20.8	14.4	3.1	14.3
Quest	4.3	21.8	28.1	21.7	16.4	3.3	15.9
SnapKV	5.2	12.8	34.6	32.6	18.3	5.0	18.1
PyramidKV	5.4	12.0	35.8	33.3	17.2	4.5	18.0
Ada-KV	6.1	12.4	34.9	34.1	18.4	4.4	18.4
FitKV (fixed B)	5.5	13.4	36.1	32.6	18.5	5.0	18.5
FitKV	6.2	13.6	36.2	31.4	19.3	5.7	18.7
N	200	224	150	300	300	200	

Table 13: Per task scores, Qwen2.5-7B, $B = 2048$. All rows use identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	TREC	TQA	PCnt	PRe	Avg.
H2O	3.7	11.0	17.7	12.9	18.6	2.0	49.0	37.0	1.3	7.0	16.0
Quest	3.5	13.7	22.5	13.8	21.0	2.2	44.0	43.0	3.6	11.5	17.9
SnapKV	5.4	15.8	34.6	25.2	27.0	4.7	42.0	54.5	2.8	14.0	22.6
FitKV (fixed B)	5.5	17.7	38.1	25.5	28.2	4.8	40.5	54.5	2.7	14.0	23.1
FitKV	5.9	26.2	44.8	27.1	32.4	6.0	69.0	58.2	2.7	15.3	28.8
N	200	200	150	200	200	200	200	200	300	300	

Table 14: Per task scores, Qwen2.5-3B, $B = 256$, on the tasks where all methods share identical examples; N is the number of examples per task.

Method	Qsp	MFQ	HQA	2WM	Mus	TREC	TQA	Avg.
H2O	16.3	22.0	15.0	20.5	3.3	58.3	40.5	25.1
Quest	17.8	26.0	14.5	20.7	4.0	55.2	47.1	26.5
SnapKV	24.0	43.0	28.4	30.5	6.2	62.5	58.2	36.1
FitKV (fixed B)	23.8	44.4	28.5	30.9	6.4	62.0	58.5	36.4
FitKV	25.9	45.3	27.1	32.9	6.0	69.0	58.2	37.8
N	200	150	200	200	200	200	200	

Table 15: Per task scores, Qwen2.5-3B, $B = 1024$, on the tasks where all methods share identical examples; N is the number of examples per task.

Method	MFQ	TREC	TQA	PCnt	PRe	Avg.
H2O	18.2	46.7	35.9	5.3	15.3	24.3
Quest	24.5	41.9	45.8	6.7	31.6	30.1
SnapKV	32.1	45.5	56.3	11.1	65.7	42.1
FitKV (fixed B)	31.5	44.0	56.3	11.1	67.0	42.0
FitKV	34.9	71.5	59.1	11.3	80.0	51.4
N	150	200	200	300	300	

Table 16: Per task scores, Qwen2.5-7B, $B = 256$, on the tasks where all methods share identical examples; N is the number of examples per task.

Method	NQA	MFQ	Mus	TREC	TQA	PCnt	PRe	Avg.
H2O	3.5	21.1	2.7	59.0	51.9	6.3	21.4	23.7
Quest	4.6	28.7	3.4	60.2	56.2	5.5	35.7	27.8
SnapKV	6.5	35.3	4.2	64.5	67.0	11.7	79.3	38.4
FitKV (fixed B)	6.3	36.1	4.4	64.5	66.7	11.7	80.0	38.5
FitKV	6.2	36.1	5.7	71.5	66.5	11.7	79.7	39.6
N	200	150	200	200	300	300	300	

Table 17: Per task scores, Qwen2.5-7B, $B = 1024$, on the tasks where all methods share identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	TREC	TQA	Avg.
H2O	2.6	5.7	16.1	10.6	11.2	6.9	50.7	58.1	20.2
Quest	3.9	5.5	15.5	9.8	10.6	6.8	48.2	60.3	20.1
SnapKV	7.3	6.1	26.4	11.3	14.2	6.5	49.5	74.9	24.5
FitKV (fixed B)	8.0	5.9	27.1	11.3	14.3	6.6	49.8	74.9	24.7
FitKV	8.6	11.9	32.6	11.8	14.0	8.5	66.0	75.1	28.6
N	200	200	150	200	200	200	300	300	

Table 18: Per task scores, Llama-3.1-8B, $B = 256$, on the tasks where all methods share identical examples; N is the number of examples per task.

Method	NQA	Qsp	MFQ	HQA	2WM	Mus	TREC	TQA	Avg.
H2O	2.8	6.9	19.4	11.0	12.2	7.0	60.0	60.0	22.4
Quest	4.5	7.3	19.3	9.7	10.8	7.7	57.6	59.4	22.0
SnapKV	9.7	9.4	29.3	11.9	13.8	7.4	62.7	74.7	27.4
FitKV (fixed B)	9.5	9.0	31.3	11.8	13.6	7.4	63.7	74.7	27.6
FitKV	8.6	12.1	32.3	11.8	14.0	8.7	66.0	75.0	28.6
N	200	200	150	200	200	200	300	300	

Table 19: Per task scores, Llama-3.1-8B, $B = 1024$, on the tasks where all methods share identical examples; N is the number of examples per task.